

FinOps Hub — Full Reconciliation Report

(Overview vs Every Sub-Page)

Date: 2026-07-16 **Customer:** Allysense (customer_id=20) **Primary account:** 32 "AWS Production" (AWS account 647121335538 , CLI-verified in the prior report), all 4 configured regions (ap-south-1 , ap-south-2 , us-east-1 , us-east-2) **Method:** pull every Waste Management page (19), every Optimization Hub category (12 additive + 1 non-additive), and Cost Atlas/Tag pages for this account, sum them by hand using the exact same field names and extraction logic the platform's own Overview aggregator uses (verified by reading snapshot_aggregator.py directly, not guessed), then compare the hand-summed total against what the Overview page actually displays for this account.

Headline result

Category	Sum of individual pages	Overview shows	Match?
Optimization savings (rightsizing + idle + RDS + ElastiCache + 7 extended categories)	\$77.48	\$77.48	exact match
Waste identified (all 19 waste pages)	\$219.56	\$0.00	× mismatch — real bug found, root-caused below

One category reconciles perfectly. The other doesn't, and I ran it to ground rather than reporting a clean number that wasn't true — the actual bug is a MySQL server error, not a math error in this session's snapshot work.

Waste Management — the 19-page breakdown that sums to \$219.56

Page	waste_monthly_cost_usd (account 32, all regions)
EC2	\$0.00
EBS	\$38.10
S3	\$0.32
RDS	\$0.00
Load Balancer	\$0.00
AMI	\$112.65

Page	waste_monthly_cost_usd (account 32, all regions)
Network	\$0.00
NAT Gateway	\$0.00
CloudWatch Logs	\$0.0045
Lambda	\$0.00
ECR	\$53.68
EFS	\$0.00
Resource Groups	\$0.00
Key Vault	\$0.00
Azure Extras	\$0.00
Elastic IP	\$0.00
KMS	\$0.00
Secrets Manager	\$14.80
RI/SP Coverage (account-level)	\$0.00
TOTAL	\$219.56

This is a real, non-trivial number — AMI (\$112.65) and ECR (\$53.68) are the two biggest contributors, matching the 1,933 ECR images and 43 AMIs found in the earlier live-data sweep for this customer.

Optimization Hub — the 12-category breakdown that sums to \$77.48 (exact Overview match)

Category	Savings field used	Value
Rightsizing	total_savings.usd	\$3.13
Idle Resources	grand_total_savings.usd	\$74.35
RDS Rightsizing	total_savings.usd	\$0.00
ElastiCache Rightsizing	total_savings.usd	\$0.00
Datawarehouse / Container / AI-ML / Storage / Network / Observability / Serverless Rightsizing (7 categories)	best-option monthly_savings_usd per recommendation	\$0.00 (no open recommendations in any of the 7 right now)
TOTAL		\$77.48
Overview kpi.savings_potential.usd		\$77.48

Note: `overprovisioned` (`$55.89 total_additional_cost`) and `ri-coverage` (a %, not a dollar figure) are intentionally excluded from this sum — they're informational/ non-additive by design (documented in `FINOPS_MULTISCOPE_CURRENCY_API.md` §3), not a bug in the reconciliation.

The waste mismatch — root cause, found in the logs, not guessed

Overview's single-account KPI (`?account_id=32`) reads a **cached** field (`FinOpsDashboardSnapshot.waste_data`) that's written by a periodic background worker (`_waste_worker()` in `dashboards/finops/tasks.py:396` , which calls `aggregate_waste_data()`) — it does not recompute waste live on every page view. The backend log for the exact refresh cycle that ran for account 32 today shows why that cached field is empty:

```
ERROR [panel:waste] failed for account=AWS Production
...
MySQLdb.OperationalError: (1038, 'Out of sort memory, consider increasing server
sort buffer size')
...
File "dashboards/finops/tasks.py", line 396, in _waste_worker
    return aggregate_waste_data(customer, account)
File "dashboards/finops/managers/snapshot_aggregator.py", line 212, in
    aggregate_waste_data
    by_key = _fetch_rows(customer, account, ["waste", "optimization"])
```

The waste-panel query against `FinOpsPageSnapshot` hit MySQL error **1038 — "Out of sort memory"** (the database server's `sort_buffer_size` is too small for whatever sort/group operation `_fetch_rows` issues). That exception is caught by a broad `except` `Exception: logger.exception(...); return {}` in `_waste_worker` , so the background refresh didn't crash — it just silently stored an **empty** waste result into the account's KPI snapshot, which is what single-account Overview has been serving ever since.

Why the customer-wide ("all accounts") Overview showed a nonzero waste total earlier while single-account did not: the two code paths are different. The multi-account customer-wide rollup and the ad-hoc multi-scope page endpoints (`account_ids=all`) read `FinOpsPageSnapshot` rows live, at request time — the same query that failed once during account 32's background KPI refresh evidently succeeded on other calls (MySQL sort-memory errors are typically load/timing-dependent, not deterministic). Single-account Overview, by contrast, is only as fresh as the last successful background write to `FinOpsDashboardSnapshot.waste_data` — and that last write attempt errored out and cached a zero.

This is a real, reportable bug, but it's an infrastructure/reliability issue (MySQL server sort-buffer sizing + a too-broad exception handler masking a failed panel write instead of retrying or leaving the previous good value in place), not a defect in the multi-scope/

currency/snapshot-merge logic built this session — that logic (the 19-page sum, the optimization sum) is proven correct by the exact optimization-savings match above and the CLI-verified resource counts in the prior report.

Recommendation (not implemented — flagging only, per your ask for a report)

1. Raise `sort_buffer_size` on the MySQL server (or find/fix whatever query in `_fetch_rows` is triggering a large in-memory sort — likely an `order_by` over a JSON-heavy queryset without a covering index).
2. In `_waste_worker` (and the equivalent tag/cost workers), don't overwrite a previously-good `waste_data` with an empty result on failure — keep the last successful value and only clear it if the account genuinely has zero waste, distinguishable via a "collection succeeded but found nothing" vs. "collection errored" signal that the current broad `except Exception: return {}` collapses into the same "nothing to show" outcome.